Day 6 Solutions:

1. Number of Common Factors

Looking at this problem, I need to count how many numbers divide both a and b evenly. Instead of checking all numbers up to the minimum of a and b, I can optimize the approach by using the Greatest Common Divisor (GCD).

Any common factor of a and b must also be a factor of gcd(a, b). So instead of checking both numbers separately, I only need to count the divisors of their GCD.

1. Find gcd(a, b) since all common factors must divide the GCD.
2. Initialize a counter to track the number of divisors.
3. Loop from 1 to gcd(a, b) (inclusive):
    - If gcd(a, b) % i == 0, then i is a common factor.
    - Increment the counter.
4. Return the total count.

   The modulo operation checks divisibility — if the remainder is 0, then i is a factor.

Code:

```
import math

class Solution:

    def commonFactors(self, a: int, b: int) -> int:

        g = math.gcd(a, b)

        count = 0

        for i in range(1, g + 1):

            if g % i == 0:

                count += 1

        return count
```

Without using libraries:

find the minimum of a and b since no common factor can exceed the smaller number

Code:

```
class Solution(object):
    def commonFactors(self, a, b):
        minval = min(a, b)
        count = 0
        for i in range(1, minval + 1):
            if a % i == 0 and b % i == 0:
                count += 1
        return count
```

Complexity

Time complexity: O(min(a,b))
We iterate from 1 to the minimum of a and b, performing constant-time modulo operations at each step.

Space complexity: O(1)
We only use two variables (minval and count) regardless of input size.

2. Find Greatest Common Divisor of Array

The problem asks for the **greatest common divisor (GCD)** of the smallest and largest numbers in the array. Since the GCD of all numbers in the array is equal to the GCD of its **minimum and maximum elements**, we only need to focus on these two values.
To efficiently compute the GCD, we use the **Euclidean Algorithm**, which repeatedly reduces the problem size using remainders.

1. Assign:
    - m as the largest element
    - n as the smallest element
2. Apply the Euclidean Algorithm:
    - While n is greater than 0:
        - Compute the remainder m % n
        - Update m to n
        - Update n to the remainder
3. When n becomes 0, m contains the GCD.
4. Return m.

Code:

```
class Solution:
    def findGCD(self, nums: List[int]) -> int:
        a = min(nums) #smallest number
        b = max(nums) #largest number


        #euclidean formula
        while(b!=0):
            temp = b
            b = a%b
            a = temp
        return a
```

By using math library

Code:

```
import math
class Solution:
    def findGCD(self, nums: List[int]) -> int:
        return math.gcd(min(nums),max(nums))
```

3. Defanging an IP Address

This problem asks us to "defang" an IP address by replacing each dot (.) with [.] to make it non-functional as a valid IP address. This is a security measure to prevent accidental clicks or automatic parsing of IP addresses in text. We need to scan through the string and replace dots while keeping all other characters unchanged.

Approach

We'll use a character-by-character replacement strategy:

1. **Character iteration**: Process each character in the IP address string
2. **Dot detection**: Check if the current character is a dot (.)
3. **Conditional replacement**: Replace dots with "[.]", keep other characters as-is
4. **String building**: Accumulate the processed characters into a result string
5. **Complete transformation**: Return the fully defanged IP address

This approach ensures all dots are properly escaped while preserving the numeric components of the IP address.

Code:

```
class Solution:
    def defangIPaddr(self, address: str) -> str:
        ans=''
        for i in address:
            if i=='.':
                ans+='[.]'
            else:
                ans+=i
        return ans
```

Code:

```
class Solution:
    def defangIPaddr(self, address: str) -> str:
        return address.replace('.','[.]')
```